

Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

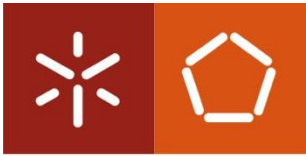
2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães

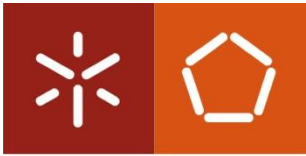


Estruturas de dados heterogéneas

Universidade do Minho

- Struct
 - Coleções de variáveis relacionadas (agregados) sob num novo tipo, nomeado de forma customizada
 - Pode conter variáveis de diferentes tipos de dados
 - Comumente usado para definir registos armazenáveis em ficheiro
 - Combinado com apontadores, pode criar listas ligadas, pilhas, filas e árvores

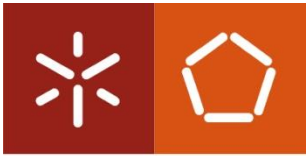
```
struct [structure tag] {  
  
    member definition;  
    member definition;  
  
    ...  
    member definition;  
} [one or more structure variables];
```



- Exemplo

```
struct card {  
    int value;  
    char name[20];  
};
```

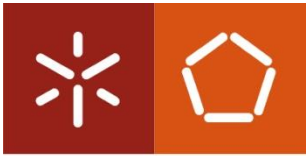
- struct introduz a definição da estrutura card
- card é o nome da estrutura e é usada para declarar variáveis do tipo dessa mesma estrutura
- card contém 2 membros de tipos distintos
 - Estes membros são o value e o name



- **struct** - informação
 - Uma estrutura não pode conter uma instância de si mesma
 - Pode conter um membro apontador para o mesmo tipo de estrutura
 - A definição de uma estrutura não reserva espaço na memória
 - Em vez disso, cria um novo tipo de dados usado para definir variáveis de estrutura

Se a definição de uma estrutura for realizada dentro de uma função ou procedimento, apenas essa função ou procedimento conhece essa definição.

Por isso, as estruturas devem ser definidas de forma a serem visíveis por todo o programa, fazendo a sua definição fora de qualquer função – em geral no início do programa ou num *header file* que se junte ao mesmo.

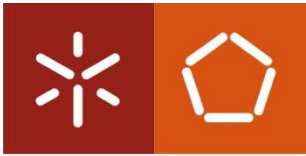


- typedef

- Cria sinónimos (*aliases*) para tipos de dados definidos anteriormente
- *typedef* pode ser usado para criar tipos com nomes mais curtos
- Exemplo:

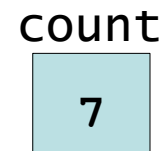
```
typedef struct Card CardType;  
CardType a;
```

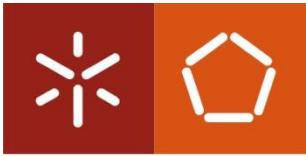
- Define um novo nome de tipo `CardType` como sinónimo de tipo `struct Card`
- `typedef` não cria novo tipo de dados – apenas a *alias*



Apontadores

- Apontadores
 - Poderoso, mas difícil de dominar
 - Pode ser usado para simular chamada por referência
 - Relacionamento próximo com *arrays* e *strings*
- Variáveis apontadoras
 - Alojamos endereços de memória como valor
 - As variáveis normais contêm um valor específico (referência direta)
 - Os apontadores contêm o endereço de uma variável que possui um valor específico (referência indireta)
 - Indireção – ato de referenciar um valor de ponteiro





- Declaração de apontadores

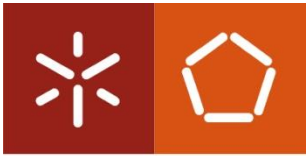
- * usado com variáveis de ponteiro

```
int *myPtr;
```

- Define um apontador para `int` (apontador do tipo `int *`)
- Para declarar vários apontadores usa-se * antes do nome da cada variável

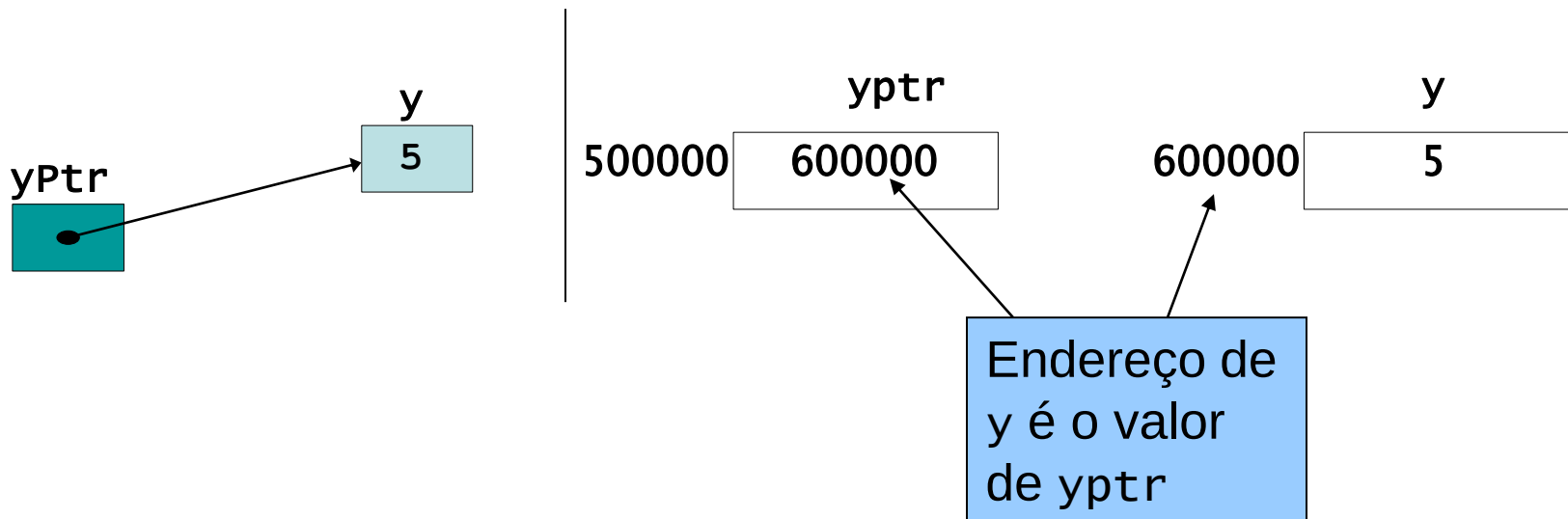
```
int *myPtr1, *myPtr2;
```

- É possível definir apontadores para qualquer tipo de dados
- Podem ser inicializados com 0, NULL, ou com um endereço de memória (&)
 - 0 ou NULL – aponta para nada (NULL é mais convencional)

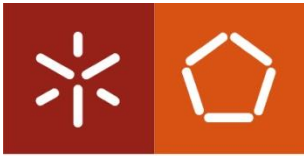


Apontadores

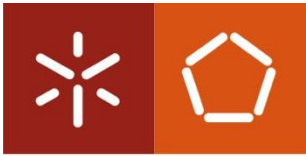
- & (operador de endereço)
 - Retorna o endereço de memória associada a uma variável
- ```
int y = 5;
int *yPtr;
yPtr = &y; /* yPtr com o endereço de y */
yPtr “aponta para” y
```







- `*` (operador de indireção/desreferenciação)
  - Retorna um sinónimo/*alias* do que seu operando ( = variável associada por referência de memória ) aponta
  - `*yptr` retorna `y` (porque `yptr` aponta para `y`)
  - `*` pode ser usado para reatribuir valores
    - Retorna a *aliase* para um objeto que esteja associado por ref. de memória
  - Desreferenciação de apontador (operando de `*`, ou seja, variável associada por `&`) tem que ser *lvalue* (i.e., não é usado para constantes)
- `*` e `&` são inversos
  - Cancelam-se mutuamente



Universidade do Minho

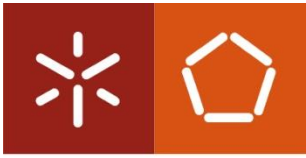
# Apontadores

- Passagem de parâmetros nas funções
  - Por valor

**Numa passagem de parâmetros por valor, são sempre enviadas para a função cópias dos valores de que esta necessita.**

- Por referência
  - Utilização dos apontadores

**EM C SÓ EXISTE PASSAGEM DE PARÂMETROS POR VALOR.**

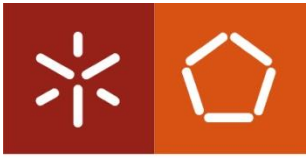


# Apontadores – passagem por referência

Universidade do Minho

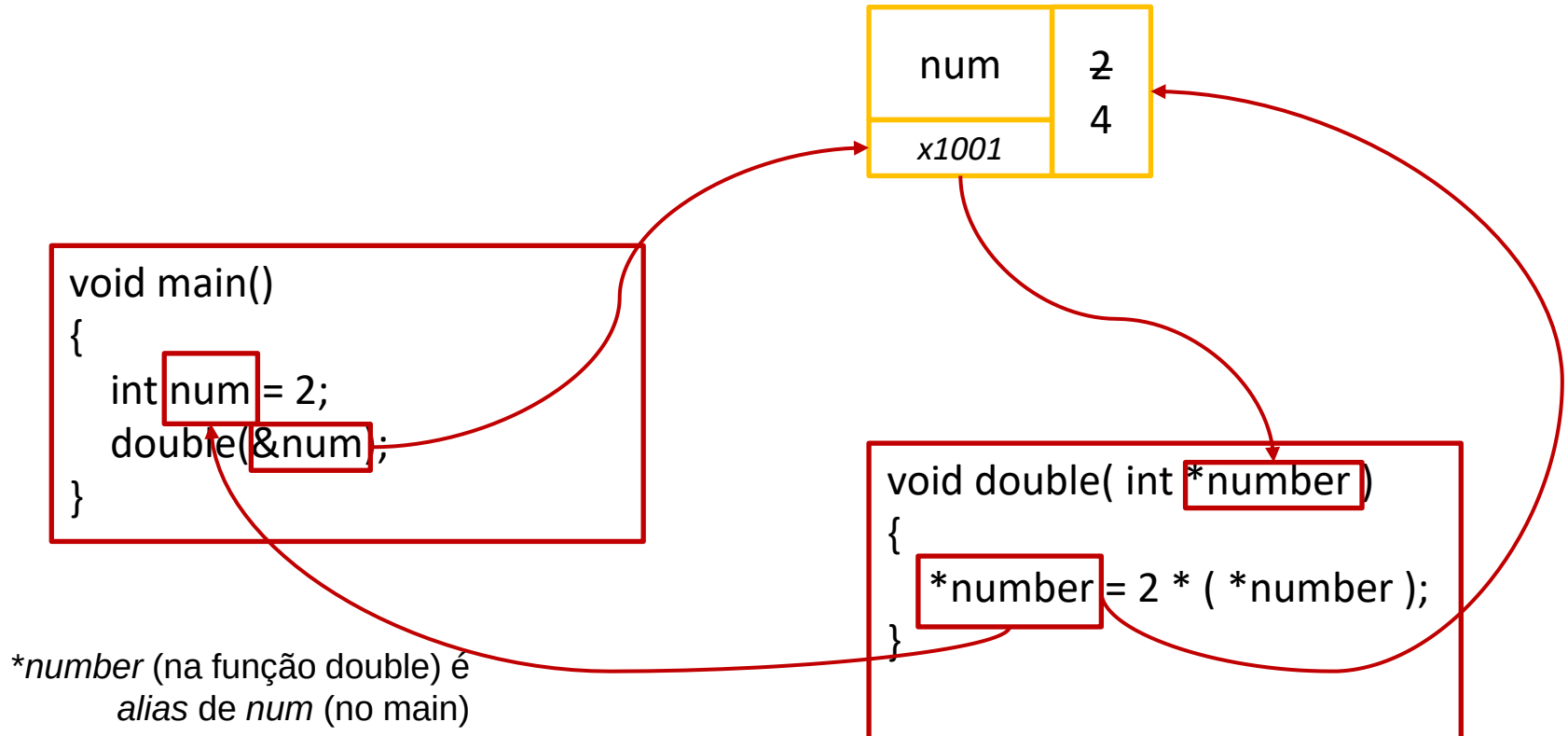
- Chamada por referência com argumentos apontadores
  - Passar o endereço de um parâmetro através do uso do operador &
  - Permite alterar a localização real na memória
  - Vetores não são passados com & porque o nome do vetor já representa o seu apontador (embora usar & não prejudique em nada)
- Operador \*
  - Usado como *alias/nickname* para variável dentro da função

```
void double(int *number)
{
 *number = 2 * (*number);
}
```
  - `*number` usado como *nickname* para a variável associada à referência que foi passada por parâmetro

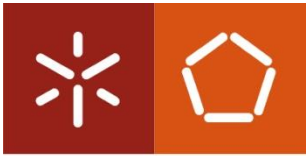


# Apontadores – passagem por referência

Universidade do Minho



**main e double partilham uma zona de memória comum, referente à mesma variável, com possibilidade de modificação do conteúdo dessa zona de memória!**



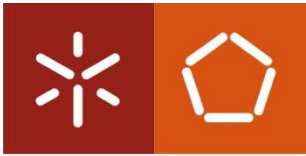
Universidade do Minho

# Apontadores

- Aritmética de apontadores
  - Incremento, decremento, diferença e comparação

**Um ponteiro para o tipo `xyz` avança sempre `sizeof(xyz)` bytes por unidade de incremento.**

**A diferença e a comparação entre ponteiros só podem ser realizadas entre ponteiros do mesmo tipo.**



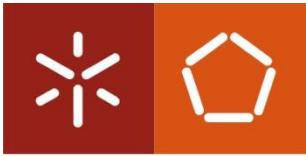
# Apontadores

- Os apontadores suportam operações aritméticas
  - Incrementar/decrementar apontador (++ ou --)
  - Adicionar/subtrair um inteiro a um apontador( + ou += , - ou -=)
  - Apontadores podem subtrair-se mutuamente
  - Resulta em deslocações pelos endereços de memória

- Apontadores e vetores

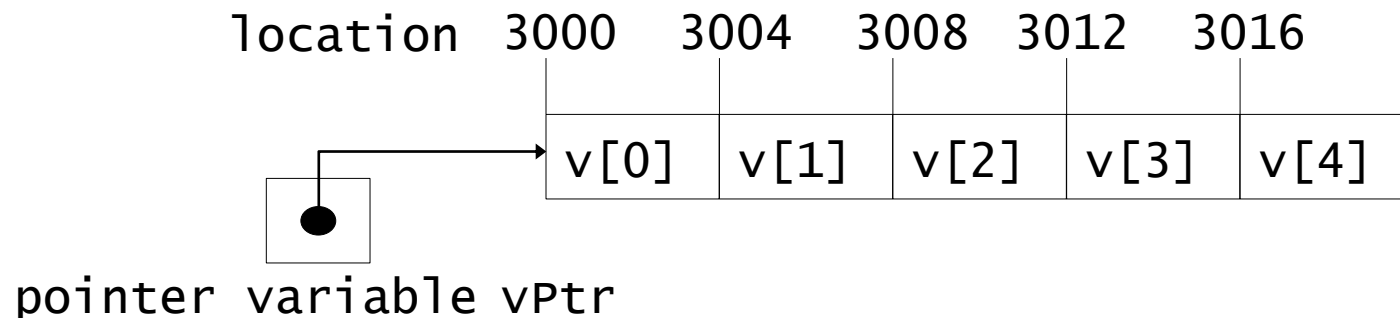
**O nome de um vetor corresponde ao endereço do seu primeiro elemento, isto é, se v for um vetor  $v == \&v[0]$ .**

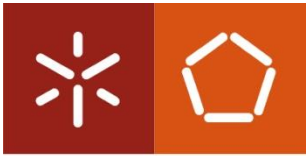
**Embora o nome de um vetor seja um ponteiro para o primeiro elemento do vetor, esse ponteiro não pode ser alterado durante a execução do programa a que pertence. Se tal fosse possível, estaríamos nos arriscando a perder o vetor previamente declarado.**



# Apontadores

- Considere-se 5 elementos num vetor de inteiros, cada um a ocupar 4 bytes (1 inteiro “gasta” 4bytes na memória)
  - vPtr aponta para o primeiro elemento `v[ 0 ]`
    - Localização 3000 da memória (`vPtr = 3000`)
  - `vPtr += 2`; colocar vPtr a 3008
    - vPtr passa a apontar para `v[ 2 ]` (incrementou 2)
    - Como a máquina “gasta” 4 bytes para cada inteiro (**int**), o endereço passa a **3008**

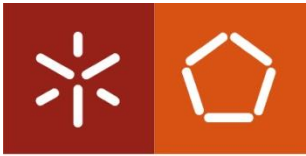




# Apontadores

- Subtração de apontadores
  - Retorna a diferença do número de elementos. Se  
`vPtr2 = &v[2];    vPtr = &v[0];`
  - Então: `vPtr2 - vPtr` irá resultar em 2 (“casinhas” de memória)
- Compara( `<`, `==`, `>` )
  - Determina qual o ponteiro que aponta para o elemento do vetor com maior/menor índice
  - Verifica se um apontador está a apontar para o “vazio” (NULL/0)
- Apontadores do mesmo tipo podem atribuir-se mutuamente
  - Se não for do mesmo tipo, um operador de conversão (*cast*) deve ser usado
  - Exceção: apontador para `void` (tipo `void *`)
    - Apontador genérico, representa qualquer tipo
    - Nenhuma conversão necessária para converter um apontador em nulo
    - Apontadores vazios não podem ser “desreferenciados”

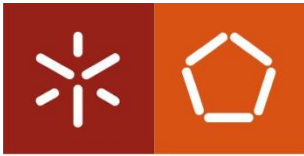




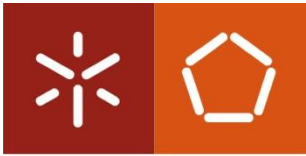
Universidade do Minho

# Apontadores

- Apontador de apontadores
  - Exemplo
    - `int **p;`

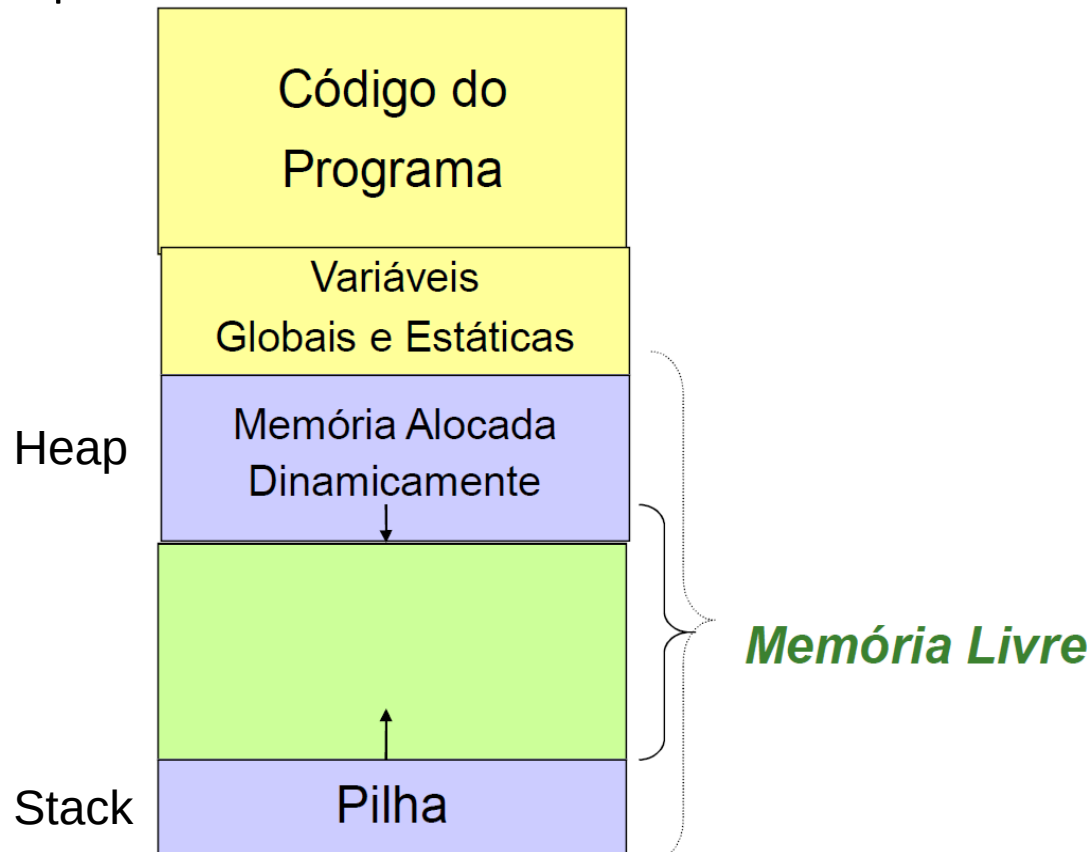


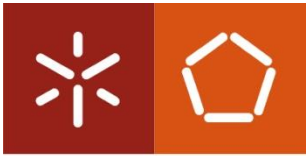
- Alocação dinâmica de memória
  - Na alocação estática de memória, os tipos de dados têm tamanho predefinido
  - A alocação estática é feita em tempo de compilação
  - Em muitas aplicações, a quantidade de memória a alocar só se torna conhecida durante a execução do programa
  - A alocação dinâmica de memória permite lidar com este problema
  - Além disso, a alocação dinâmica permite aumentar ou até diminuir a quantidade de memória alocada
  - A alocação dinâmica é feita em tempo de execução
  - O espaço alocado dinamicamente permanece reservado até que seja explicitamente libertado pelo programa. Por isso, podemos alocar dinamicamente um espaço de memória numa função e acedê-lo noutra.



# Apontadores

- Alocação dinâmica de memória
  - A região de memória utilizada para alocação dinâmica é conhecida como heap.





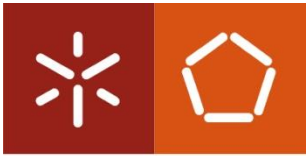
# Apontadores

- Alocação dinâmica de memória
  - A alocação dinâmica em C é gerida pelas funções malloc, calloc, realloc e free, que estão na biblioteca stdlib
  - Não esquecer de incluir a biblioteca através de
    - #include <stdlib.h>
  - **malloc**
    - A função malloc() serve para alocar memória e tem o seguinte protótipo:

```
void *malloc (unsigned int num);
```

- Dado o número de bytes que queremos alocar (num), ela aloca na memória **num** bytes contíguos e retorna um apontador **void\*** para o primeiro byte alocado.
- O apontador void\* pode ser atribuído a qualquer tipo de apontador via *type cast*.

```
char *p;
p = (char *) malloc(1000);
```



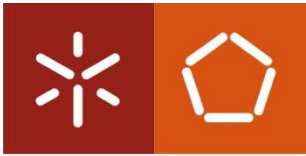
- Alocação dinâmica de memória

- **malloc**

- Operador **sizeof()**
      - Retorna o número de bytes de um dado tipo de dado. Ex.: int, float, char, struct...
    - Alocar espaço para 50 inteiros:

```
int *p;
p = (int *) malloc(50*sizeof(int));
```

- A variável p agora pode ser vista como um array com 50 inteiros
      - Se não houver memória suficiente para alocar a memória requisitada, a função malloc() (assim como a função calloc()) retorna um apontador nulo



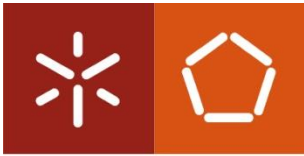
- Alocação dinâmica de memória

- malloc

- Por isso, deve sempre ser verificado, depois de usar a função malloc() ou calloc(), se o apontador está a NULL antes de realizar as operações sobre o espaço alocado.

```
int main(){
 int *p;
 p = (int *) malloc(5*sizeof(int));
 if(p == NULL){
 printf("Erro: Memoria Insuficiente!\n");
 system("pause");
 exit(1);
 }
 int i;
 for (i=0; i<5; i++){
 printf("Digite o valor da posicao %d: ",i);
 scanf("%d",&p[i]);
 }

 return 0;
}
```



- Alocação dinâmica de memória

- **calloc**

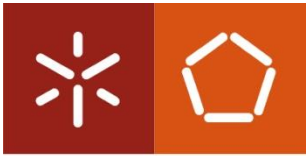
- A função `calloc()` também serve para alocar memória, mas possui um protótipo um pouco diferente:

```
void *calloc (unsigned int num, unsigned int size);
```

- A função aloca uma quantidade de memória igual a **num** \* **size**, isto é, aloca memória suficiente para um vetor de **num** objetos de tamanho **size**.

```
int main(){
 //alocação com malloc
 int *p;
 p = (int *) malloc(50*sizeof(int));
 if(p == NULL){
 printf("Erro: Memoria Insuficiente!\n");
 }
 //alocação com calloc
 int *p1;
 p1 = (int *) calloc(50, sizeof(int));
 if(p1 == NULL){
 printf("Erro: Memoria Insuficiente!\n");
 }

 return 0;
}
```



- Alocação dinâmica de memória

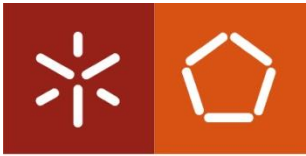
- **realloc**

- A função `realloc()` serve para realocar memória e tem o seguinte protótipo:

```
void *realloc (void *ptr, unsigned int num);
```

- A função modifica o tamanho da memória previamente alocada e apontada por **\*ptr** para aquele especificado por **num**.
      - O valor de **num** pode ser maior ou menor que o original.
      - Um apontador para o bloco é devolvido porque **realloc()** pode precisar mover o bloco para aumentar o seu tamanho.
      - Se isso ocorrer, o conteúdo do bloco antigo é copiado para o novo bloco, e nenhuma informação é perdida.

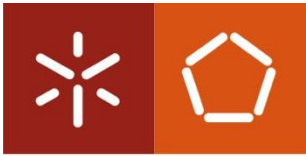




- Alocação dinâmica de memória
  - realloc

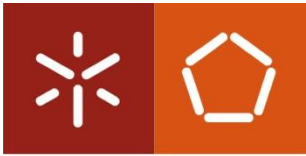
```
int main(){
 int i;
 int *p = malloc(5*sizeof(int));
 for (i = 0; i < 5; i++){
 p[i] = i+1;
 }
 for (i = 0; i < 5; i++){
 printf("%d\n",p[i]);
 }
 printf("\n");
 //Diminui o tamanho do array
 p = realloc(p,3*sizeof(int));
 for (i = 0; i < 3; i++){
 printf("%d\n",p[i]);
 }
 printf("\n");
 //Aumenta o tamanho do array
 p = realloc(p,10*sizeof(int));
 for (i = 0; i < 10; i++){
 printf("%d\n",p[i]);
 }

 return 0;
}
```

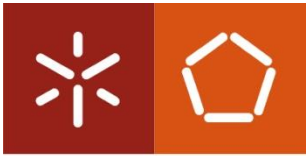


- Alocação dinâmica de memória
  - **realloc**
    - Se **\*ptr** for nulo, aloca **num** bytes e devolve um apontador (igual a **malloc**);
    - Se **num** é zero, a memória apontada por **\*ptr** é libertada (igual a **free**).
    - Se não houver memória suficiente para a alocação, é devolvido **NULL** e o bloco original é deixado inalterado.
  - **free**
    - Quando alocamos memória dinamicamente é necessário libertá-la quando ela não for mais necessária. Para isto existe a função **free()** cujo protótipo é:

```
void free(void *p);
```



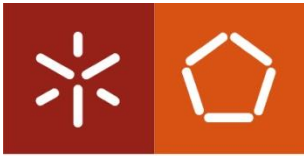
- Alocação dinâmica de memória
  - **free**
    - Basta passar para `free()` o apontador que aponta para o início da memória a desalocar/libertar.
    - Como é que o programa vai saber quantos bytes devem ser libertados?
      - Quando se aloca memória, o programa guarda o número de bytes alocados numa "tabela de alocação" interna.



- Alocação dinâmica de memória

- **free**

```
main (void)
{
 int *p;
 int a;
 ...
 p=(int *)malloc(a*sizeof(int));
 if (!p)
 {
 printf("** Erro: Memoria Insuficiente **");
 exit(1);
 }
 ...
 free(p);
 ...
}
```



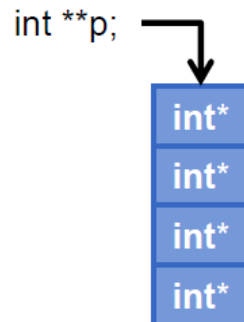
# Apontadores

- Alocação dinâmica de memória
  - Para alocarmos arrays com mais de uma dimensão, utilizamos o conceito de “apontador para apontador”.
  - Ex.: `int **p2; int ***p3;`
  - Para cada nível do apontador, fazemos a alocação de uma dimensão do array.

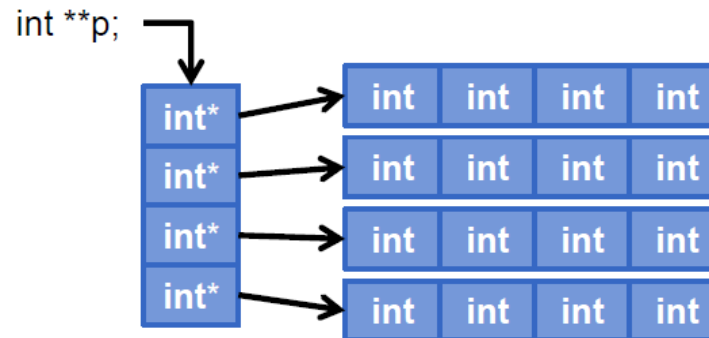
```
p = (int**) malloc(N*sizeof(int*));

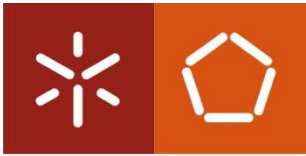
for (i = 0; i < N; i++){
 p[i] = (int *)malloc(N*sizeof(int));
}
```

1º malloc:  
cria as linhas

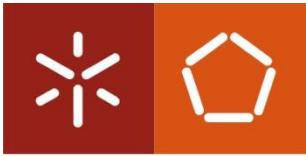


2º malloc:  
cria as colunas





- Alocação dinâmica de memória
  - Assim como os tipos básicos, também é possível fazer a alocação dinâmica de estruturas.
  - As regras são exatamente as mesmas para a alocação de uma **struct**.
  - Podemos fazer a alocação de
    - uma única **struct**
    - um array de **structs**
  - Para alocar uma única struct
    - Um apontador para struct receberá o resultado do malloc()
    - Utilizamos o operador seta para aceder aos campos da estrutura apontada pelo apontador
    - Usamos free() para libertar a memória alocada



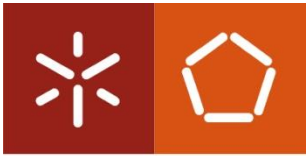
- Alocação dinâmica de memória

```
struct cadastro{
 char nome[50];
 int idade;
};

int main(){
 struct cadastro *cad = (struct cadastro*) malloc(sizeof(struct cadastro));
 strcpy(cad->nome, "Maria");
 cad->idade = 30;

 free(cad);

 return 0;
}
```



- Alocação dinâmica de memória
  - Para alocar um array de structs
    - Um apontador para struct receberá o resultado do malloc() (também pode ser usado o calloc())
    - Utilizamos os [] para aceder a cada uma das posições do vetor
    - Usamos free() para libertar a memória alocada

```
struct cadastro{
 char nome[50];
 int idade;
};

int main(){
 struct cadastro *vcad = (struct cadastro*) malloc(10*sizeof(struct cadastro));

 strcpy(vcad[0].nome, "Maria");
 vcad[0].idade = 30;

 strcpy(vcad[1].nome, "Cecilia");
 vcad[1].idade = 10;

 strcpy(vcad[2].nome, "Ana");
 vcad[2].idade = 10;

 free(vcad);

 return 0;
}
```